Langage Interprété

Gestion des dépendances et environnements

1ère année Cycle ingénieur ISEN – Lille

2026/2027

Abderrahmane Kasmi

Séquence 4

Gestion des dépendances et environnements

01 **Dépendances et résolution de versions**

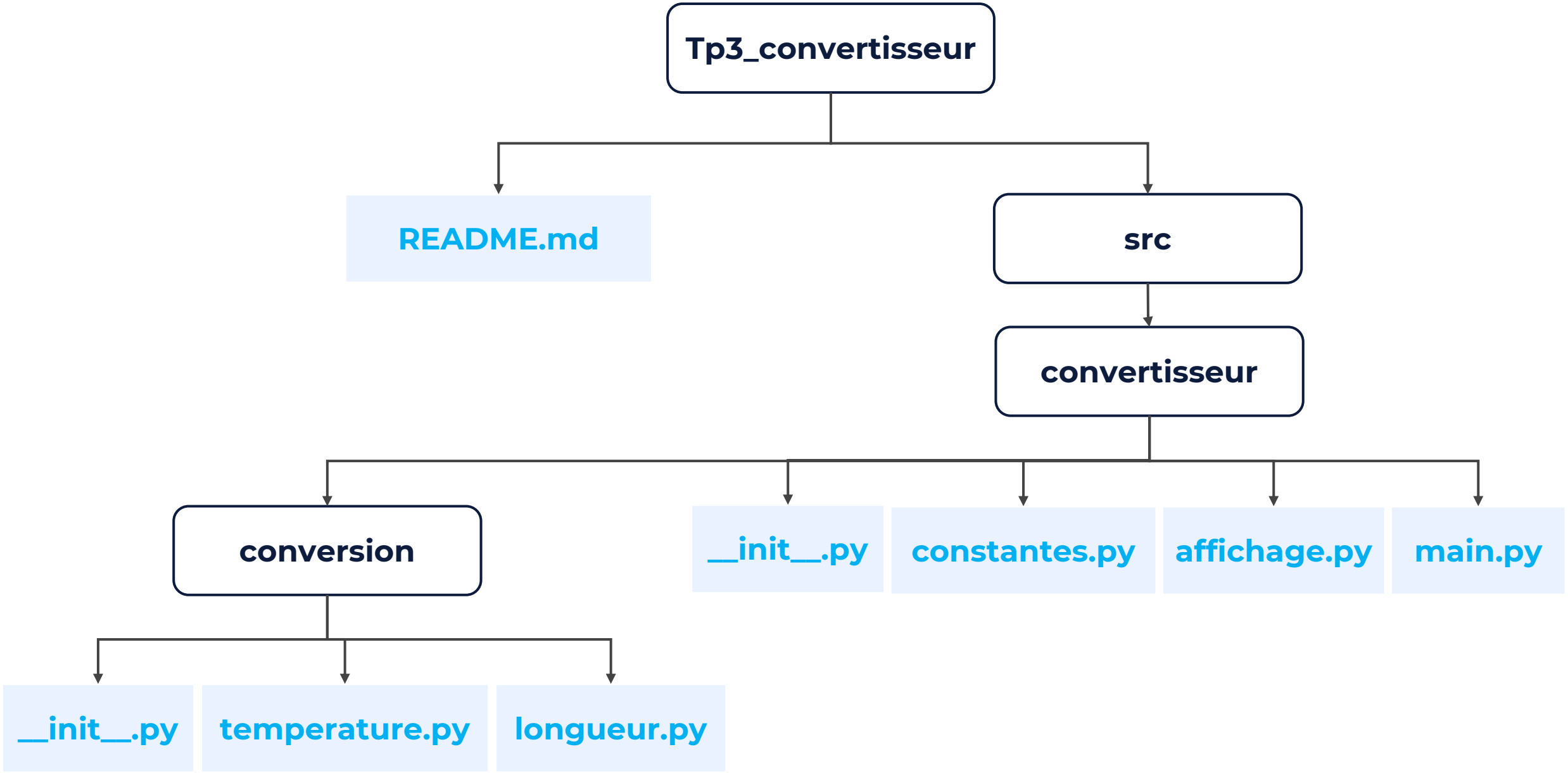
02 **Environnements virtuels**

03 **Fichier de description de projet**

04 **Arguments en ligne de commande**

00

TP3 - Pistes de correction


Tp3_convertisseur
README.md
src
convertisseur
conversion
__init__.py
constantes.py
affichage.py
main.py
__init__.py
temperature.py
longueur.py

01

Dépendances et résolution de versions

Les dépendances

C’est une bibliothèque externe dont un projet (ou une autre bibliothèque) a besoin pour fonctionne, on distingue:

- **Les dépendances directes :** celles que le projet déclare lui-même (on l’installe avec `pip install`)
- **Les dépendances transitives :** celles dont dépendent à leur tour les dépendances directes (par exemple, requests dépend de urllib3)

Les versions

Une version est un numéro qui identifie un état précis d'une bibliothèque à un moment donné. Chaque fois que les auteurs publient des changements (correction, amélioration, nouveauté), ils publient une nouvelle version. Leur format général est: **MAJEUR.MINEUR.CORRECTIF** (par exemple 2.34.1)

- **MAJEUR :** Changements **incompatibles** : du code qui marchait peut casser
- **MINEUR :** Nouvelles fonctionnalités, sans rien casser
- **CORRECTIF :** Corrections de bugs, sans rien casser

02

Environnements virtuels

Installation de bibliothèques

Un module (ou package) installé avec `pip install` ou qui existe par défaut (bibliothèque standard) est automatiquement placer dans le dossier de python

- **`Lib`** pour les bibliothèques standard
- **`Lib/site-packages`** pour les bibliothèques tierces

Problématique: comment gérer deux projets qui dépendent de la même bibliothèque mais avec deux versions différentes

Environnement virtuel

Un environnement virtuel est un dossier (souvent `.venv/`) contenant :

- une copie ou un lien vers l'interpréteur Python
- son propre dossier `site-packages`, où s'installent les bibliothèques du projet

Activation: activer l'environnement modifie le `PATH` du terminal pour que `python` et `pip` désignent ceux de l'environnement

uv

Un gestionnaire de paquets moderne, il permet de :

- Gérer les paquets (installation, mise à jour, désinstallation)
- Gérer les environnements virtuels
- Assurer la reproductibilité des projets

Pourquoi uv? souvent 10 à 100 fois plus que `pip`, compatible avec `pip`, compatible avec les fichiers de standard actuel.

Commandes de base

- **Création de projet**

```
$ uv init nom-du-projet
```

- **Installation de bibliothèque (crée l'environnement si besoin)**

```
$ uv add nom-du-paquet
```

- **Désinstallation de bibliothèque**

```
$ uv remove nom-du-paquet
```

Commandes de base

- **Création de l'environnement virtuel**

```
$ uv venv
```

- **Executer un fichier (sans avoir besoin d'activation de l'environement)**

```
$ uv run main.py
```

- **Commandes compatibles avec `pip`**

```
$ uv pip …
```

03

Fichier de description de projet

Reproductibilité

Un projet reproductible est un projet qui permet d'obtenir exactement **le même fonctionnement** et les mêmes résultats, sur **n'importe quelle machine** et à **n'importe quel moment**.

Pour cela, **une description détaillée des dépendances** du projet doit être fournie, afin de garantir que le programme s'exécutera dans **le même environnement d'exécution que l'original**

`pyproject.toml`

C’est le fichier de configuration d’un projet Python : il décrit **le projet** (nom, version, version de Python requise) et **liste les dépendances** dont il a besoin pour fonctionner

```toml
[project]
name = "convertisseur"
version = "0.1.0"
requires-python = "≥3.13"
dependencies = [
    "numpy≥2.0",
]
```

`uv.lock`

C’est le fichier de verrouillage généré par uv.Il enregistre la **liste exacte de toutes les dépendances du projet** (directes et indirectes), avec **leur version précise**, **leur provenance** et leurs empreintes (hashes) de vérification

```
version = 1
revision = 3
requires-python = "≥3.13"

[[package]]
name = "convertisseur"
version = "0.1.0"
source = { virtual = "." }
dependencies = [
    { name = "numpy" },
]

[package.metadata]
requires-dist = [{ name = "numpy", specifier = "≥2.0" }]

[[package]]
name = "numpy"
version = "2.5.3"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/ … /numpy-2.5.3.tar.gz", hash =
"sha256:df2d5874 … " }
wheels = [
    { url = "https://files.pythonhosted.org/ … /numpy-2.5.3-cp313- … -
macosx_10_13_x86_64.whl", hash = "sha256:66a78fe4 … " },
    { url = "https://files.pythonhosted.org/ … /numpy-2.5.3-cp313- … -
manylinux_2_28_x86_64.whl", hash = "sha256: … " },
    { url = "https://files.pythonhosted.org/ … /numpy-2.5.3-cp313- … -
win_amd64.whl", hash = "sha256: … " },
    # … une ligne par système et version de Python
]
```

Autres commandes

- **Créer un environnement en se basant sur `uv.lock`**

```
$ uv sync
```

- **Mettre à jour `uv.lock` en se basant sur `pyproject.toml`**

```
$ uv lock
```

- **Afficher les dépendances de projet sous forme d'arbre**

```
$ uv tree
```

Reproduire un environnement

Pour un projet reproductible, on doit fournir :

- **Le code source**
- **pyproject.toml**
- **uv.lock**
- **README.md**
- ~~**.venv**~~

04

Arguments en ligne de commande

Motivation

Dans le TP3, on a créé un outil de conversion de température

```
def celsius_vers_fahrenheit(c):
    return (c * 9 / 5) + 32


if __name__ == "__main__":
    c = 25
    print(celsius_vers_fahrenheit(c))
```

Et si on veut utiliser le même outil mais pour une autre valeur ?

Motivation

Les arguments en ligne de commande permettent d'utiliser le même outil pour différent paramètres:

```
$ python hello.py --valeur 30
```

```
$ python hello.py --valeur -10
```

Input()

Input() permet de suspendre l'exécution d'un programme et de recevoir une entrée de la part de l'utilisateur

```
def celsius_vers_fahrenheit(c):
    return (c * 9 / 5) + 32


if __name__ == "__main__":
    c = float(input("Veuillez entrer une valeur:"))
    print(celsius_vers_fahrenheit(c))
```

Le problème est que l'exécution est suspendue en attendant une valeur de la part de l'utilisateur

Argparse

- **Création de parser**

```
import argparse

parser = argparse.ArgumentParser(description="Convertit une température
entre Celsius et Fahrenheit.")
```

- **Ajout d’un argument**

```
parser.add_argument("valeur", type=float, help="température à convertir")
```

- **Récupération des arguments**

```
Args = parser.parse_args()
```

Paramètres des arguments

- **Nom:** Sans tiret, l'argument est positionnel : obligatoire, identifié par sa place. Avec `--`, c'est une option : identifiée par son nom, généralement facultative
- **Nom court:** Alias abrégé d'une option, avec un seul tiret. On peut donner les deux : `"-v"`, `"--verbose"`
- **Type:** Fonction de conversion appliquée à la chaîne reçue. Sans type, tout reste une chaîne (`str`)
- **Default:** Valeur utilisée si l'option n'est pas fournie
- **Choices:** Liste des valeurs autorisées. Toute autre valeur provoque une erreur
- **Help:** Texte affiché dans `--help` pour décrire l'argument